

Shreyaa Kashyap

28-7-2026

7-8-2026

Robotics Camp 2026 Capstone

Introduction

To demonstrate the skills in both circuit building and computer science that I have acquired over an 8 week Robotics Camp, I have created a Capstone Project with the premise being to showcase different facets of robotics in one fully-functioning system. Throughout my experience with digital electronics, I have worked with breadboards and both input and output devices with the completion of 50+ SunFounder Pico 2W projects.

Page Layout

I have designed this documentation page with the steps used to create an original project in mind. I used the CARD-BTP method with CARD (Concept, Analysis, Requirements, and Design) as the planning steps and BTP (Build, Test, and Present) as the execution steps.

More detailed information can be found throughout this documentation.

Resources Provided

For this Capstone Project I will be using MicroPython as the central programming language and Thonny IDE as the development environment. The components which I will be using are part of the SunFounder Pico 2W Kit.

A Reminder

As a reader, please note that parts of this document have intentionally been left blank as they are still a work-in-progress. As I continue to move forward in my work, these pages will be promptly updated.

Planning Steps

The development of this Capstone project has been separated into two steps, one of which is dedicated to Planning. This step consists of the Concept Analysis Requirements Design portion of CARD-BTP.

Concept

The first part of a plan is developing an idea. I represented this stage by making a list of candidate ideas and choosing the final one based on feasibility and how well it incorporated the concepts that I learned.

Analysis

Analysis is breaking up a goal into small manageable chunks such as thinking about the materials necessary as well as looking for alternative pathways to reach a similar result.

Requirements

This step is unique to the others as it can and should run in parallel to the rest of the planning phase. Thinking about requirements is a requisite for any successful project so naturally I included it in this phase.

Design

The final part of planning is coming up with an effective design. For this I included a detailed circuit diagram and a more general build layout.

Execution Steps

The second series of steps is dedicated to the successful execution of the planned project and the presentation of the cumulative results.

Build

Building the prototype includes both constructing a functioning circuit and functioning code. The prototype would then be subjected to various tests as part of the next step.

Test

After the prototype has been built, it will be put through a series of tests in this stage to ensure proper functionality. This step runs concurrently with the build step.

Present

The demonstration of a working Capstone project and the work behind it would take place during the Presentation step. Since the documentation itself is part of presenting, this step has been omitted.

Pico 2W SunFounder Kit

My primary resources for this project are found in the [Pico 2W SunFounder Kit](#) with an additional [OLED Display Module](#)

Many of the online resources I used throughout this camp (excluding the Capstone) can be found at [the official SunFounder docs page](#)

Concept Overview

For this project, I had two classes of ideas in mind: Practical and Entertainment. The Practical Ideas would have real-world uses while the Entertainment Ideas would function more like games and fun displays.

Candidate Ideas

Practical

- Car sensor, similar to the one in front of modern cars used to assist drivers in maintaining a safe distance between the vehicle and other objects
- Automatic plant waterer where users can set a time for their plant to be supplied with water daily
- A temperature detector which triggers a small fan when the detected temperature exceeds a certain level
- A security system using an RFID module which displays a secret message only if the right tag interacts with it
- An alarm clock set by the user
- A way to display the power that a batter connected provides

Entertainment

- A ping-pong game where users can choose between playing on auto mode or against another player
- An LED strip that can have its colors altered with a WiFi connected service
- A music player which converts songs found online into a format that can be played by a passive buzzer
- Manually turning a knob to change the pitch of a sound
- Pac-man
- Atari Breakout

Final Idea

My final idea is a Ping Pong game from the Entertainment category. I decided to use this as my project because of its feasibility as well as how it uses complex mechanics making it perfect for a final demonstration of what I have learned in this Robotics Camp.

Analysis Overview

The next step in planning my Capstone is to analyze the goal that I have given myself which is to build a Ping-Pong game. This involves drafting a list of materials and finding out the best possible route to a fully functioning ping-pong game.

Materials Needed

Materials	Amount
-----------	--------

Breadboard	1
Pico	1
Jumper Wires	Several
OLED Module	1
Buttons	4
10KΩ Resistors	4
Infrared Receiver	1
Infrared Controller	1

Alternative Materials

Alternate 1

Materials	Amount
Breadboard	1

Pico	1
Jumper Wires	Several
OLED Module	1
Potentiometer	2
220Ω Resistors	2
Infrared Receiver	1
Infrared Controller	1

Alternate 2

Materials	Amount
Breadboard	1
Pico	1
Jumper Wires	Several
OLED Module	1

Potentiometer	2
220Ω Resistors	2
Button	1
10KΩ Resistor	1

Alternate 3

Materials	Amount
Breadboard	1
Pico	1
Jumper Wires	Several
OLED Module	1
Potentiometer	2
220Ω Resistors	2
Button	1

10K Ω Resistor	1
LCD Display	1

Requirements Overview

Every project must meet certain requirements in order for it to be considered successful. Therefore, it is essential to have the requirements for this project specified early on in the plan stage.

Characteristics of a Ping-Pong Game

Display

- Show the playing field.
- Display two paddles.
- Display a moving ball.
- Show player scores.

Paddle Controls

- Player 1 can move the paddle up and down.

- Player 2 can move the paddle up and down (or be bot-controlled).
- Prevent paddles from moving off-screen.

Movement

- The ball starts in the center.
- The ball moves continuously.
- The ball bounces off the top and bottom edges.
- The ball changes direction after hitting a paddle.

Collision Detection

- Detect collisions between:
 - Ball and paddles
 - Ball and screen boundaries
- Update the ball's direction accordingly.

Scoring

- Award one point when the opponent misses the ball.
- Reset the ball to the center after each point.
- Display updated scores.

Game End

- End the game when a player reaches a target score (e.g., 5 or 10).
- Display the winner

Specifications

A ping pong game must show the proper layout, enable paddle movement, make the ball move in a realistic manner, detect collisions between both paddles and walls, award points where needed, and detect the winner.

Design Overview

The design step is used to give both the project creator and the project review an idea of how to build the game. In this documentation, I have provided both a detailed circuit diagram and a more general layout.

Circuit Diagram

The circuit diagram can be found [here](#)

Build Layout

The basic layout involves an OLED screen with VCC and Grounds connected accordingly as well as SDA and SCL pins with the GPIO pins and channels connected accordingly. 4 buttons will be connected in parallel with the GPIO pins, power, and ground at the correct positions and with 10K Ω resistors in between. Finally, the infrared sensor will be connected within the circuit with 220 Ω resistors regulating current flow.

Build Overview

The first execution step involves the construction of a circuit and the programming which will later allow it to function completely.

Prototype

A link to one of my early prototypes of this program can be found [here](#). Keep in mind, it is one of my alternative ideas which I ended up using for this project.

Code

My code for this Capstone project has been refined several times to make a functioning Ping Pong game. An example of one of my early drafts for [CAPSTONE.py](#) can be found here:

```
from machine import Pin, I2C, ADC

import time

import random

from ssd1306 import SSD1306_I2C

from ir_rx.nec import NEC_8


i2c_left = I2C(0, scl=Pin(13), sda=Pin(12), freq=400000)

i2c_right = I2C(1, scl=Pin(15), sda=Pin(14), freq=400000)

oled_left = SSD1306_I2C(128, 64, i2c_left)

oled_right = SSD1306_I2C(128, 64, i2c_right)

joystick = ADC(27)

ir_pin = Pin(17, Pin.IN)


game_started = False
```

```
ai_mode = False
```

```
winner = 0
```

```
def ir_callback(data, addr, ctrl):
```

```
    global game_started
```

```
    global ai_mode
```

```
    global winner
```

```
    print(f>Data: {data}/{type(data)}, Address: {addr}, Control: {ctrl}")
```

```
    if data >= 0:
```

```
        ir_button = data
```

```
        if game_started:
```

```
            if winner == 0 and not ai_mode:
```

```
                move_paddle2_manual(ir_button)
```

```
        else:
```

```
            print(f"Game: {game_started}, {data}")
```

```
            if ir_button == MODE:
```

```
                ai_mode = not ai_mode
```

```
                draw_menu()
```

```
            elif ir_button == OK:
```

```
                reset_game()
```

```
game_started = True
```

```
ir = NEC_8(ir_pin, ir_callback)
```

```
MODE = 0x46
```

```
OK = 0x45
```

```
PLUS = 0x09
```

```
MINUS = 0x15
```

```
WIDTH = 256
```

```
HEIGHT = 64
```

```
PADDLE_HEIGHT = 12
```

```
PADDLE_WIDTH = 2
```

```
BALL_SIZE = 2
```

```
score1 = 0
```

```
score2 = 0
```

```
paddle1_y = 26
```

```
paddle2_y = 26
```

```
ball_x = 128
```

```
ball_y = 32
```

```
ball_dx = 1
```

```
ball_dy = 1
```

```
def clear_displays():
```

```
    oled_left.fill(0)
```

```
    oled_right.fill(0)
```

```
def draw_pixel(x, y):  
    if x < 0 or x >= WIDTH:  
        return  
  
    if y < 0 or y >= HEIGHT:  
        return  
  
    if x < 128:  
        oled_left.pixel(x, y, 1)  
  
    else:  
        oled_right.pixel(x - 128, y, 1)  
  
def draw_rect(x, y, w, h):  
    for yy in range(y, y+h):  
        for xx in range(x, x+w):  
            draw_pixel(xx, yy)  
  
def draw_text(x, y, text):  
    if x < 128:  
        oled_left.text(text, x, y, 1)  
  
    else:  
        oled_right.text(text, x-128, y, 1)
```

```

def show():

    oled_left.show()

    oled_right.show()


# -----

# DRAW FULL PLAYFIELD

# -----

def draw_field():

    clear_displays()

    # middle line

    for y in range(0, 64, 4):

        draw_pixel(128, y)


    # paddles

    draw_rect(4, paddle1_y, PADDLE_WIDTH, PADDLE_HEIGHT)


    draw_rect(250, paddle2_y, PADDLE_WIDTH, PADDLE_HEIGHT)

    # ball

    draw_rect(ball_x, ball_y, BALL_SIZE, BALL_SIZE)


    # scores

    draw_text(55, 0, str(score1))

    draw_text(183, 0, str(score2))

    show()

```

```

# -----

# INPUT FUNCTIONS

# -----


def read_joystick():

    value = joystick.read_ul6()

    # joystick centered around ~32768

    if value < 20000:

        return -1


    if value > 45000:

        return 1


    return 0


# -----

# PADDLE CONTROL

# -----


def move_paddle1():

    global paddle1_y


    direction = read_joystick()

    paddle1_y += direction * 2


    if paddle1_y < 0:

```

```

    paddle1_y = 0

    if paddle1_y > HEIGHT - PADDLE_HEIGHT:
        paddle1_y = HEIGHT - PADDLE_HEIGHT

def move_paddle2_manual(ir_button):
    print("move_paddle2_manual() was called")
    global paddle2_y

    if ir_button == PLUS:
        print("Plus detected!")
        paddle2_y -= 3

    elif ir_button == MINUS:
        print("Minus detected!")
        paddle2_y += 3

    if paddle2_y < 0:
        paddle2_y = 0

    if paddle2_y > HEIGHT - PADDLE_HEIGHT:
        paddle2_y = HEIGHT - PADDLE_HEIGHT

# -----
# AI PADDLE

```

```

# -----

def move_paddle2_ai():

    global paddle2_y

    target = ball_y - (PADDLE_HEIGHT // 2)

    if paddle2_y < target:

        paddle2_y += 1

    elif paddle2_y > target:

        paddle2_y -= 1

    if paddle2_y < 0:

        paddle2_y = 0

    if paddle2_y > HEIGHT - PADDLE_HEIGHT:

        paddle2_y = HEIGHT - PADDLE_HEIGHT

# -----

# MENU SCREEN

# -----

def draw_menu():

    clear_displays()

    draw_text(50, 9, "PING")

    draw_text(178, 9, "PONG")

```

```

if ai_mode:

    draw_text(5, 30, "VS AI")

else:

    draw_text(5, 30, "VS PLAYER")


draw_text(5, 50, "POWER TO START")

draw_text(130, 30, "'MODE' TO CHANGE")

draw_text(130, 50, "GAMEMODE (P2/AI)")


show()


# -----
# BALL FUNCTIONS
# -----


def reset_ball():

    global ball_x

    global ball_y

    global ball_dx

    global ball_dy


    ball_x = 128

```

```
ball_y = 32

ball_dx = random.choice([-1, 1])
ball_dy = random.choice([-1, 1])

def reset_game():

    global score1

    global score2

    global paddle1_y

    global paddle2_y

    global winner

    score1 = 0

    score2 = 0

    paddle1_y = 26

    paddle2_y = 26

    winner = 0

    reset_ball()

# -----
# BALL COLLISION
# -----

def move_ball():

    global ball_x

    global ball_y
```

```
global ball_dx

global ball_dy

ball_x += ball_dx * 2

ball_y += ball_dy * 2

# Top and bottom walls

if ball_y <= 0:

    ball_y = 0

    ball_dy *= -1

if ball_y >= HEIGHT - BALL_SIZE:

    ball_y = HEIGHT - BALL_SIZE

    ball_dy *= -1

# Player 1 paddle

if ball_x <= 6:

    if (paddle1_y <= ball_y <= paddle1_y + PADDLE_HEIGHT):

        ball_x = 6

        ball_dx *= -1

# Player 2 paddle

if ball_x >= 246:

    if (paddle2_y <= ball_y <= paddle2_y + PADDLE_HEIGHT):

        ball_x = 246
```

```
        ball_dx *= -1

# -----

# SCORE HANDLING

# -----

def check_score():

    global score1

    global score2

    global winner

    if ball_x < 0:

        score2 += 1

        reset_ball()

    if ball_x > WIDTH:

        score1 += 1

        reset_ball()

    if score1 >= 5:

        winner = 1

    if score2 >= 5:

        winner = 2

# -----

# WIN SCREEN

# -----

def draw_winner():
```

```

clear_displays()

if winner == 1:

    draw_text(20, 25, "P1 WINS")

else:

    draw_text(150, 25, "P2 WINS")

show()

# -----
# MAIN GAME LOOP
# -----

def main():

    global game_started

    while True:

        game_started = False

        draw_menu()

        reset_game()

        while game_started and winner == 0:

            # Player 1

            move_paddle1()

```

```
# Player 2

if ai_mode:

    move_paddle2_ai()

move_ball()

check_score()

draw_field()

time.sleep(0.02)

if winner != 0:

    draw_winner()

    time.sleep(5)

    reset_game

    game_started = False

if __name__ == "__main__":

    main()
```

Testing Overview

The testing, refining, and perfecting of the prototype is a part of the test stage. The test stage could be considered as part of the build stage when discussing the executional steps' sequences.

The testing procedure and the final build can be found in the tabs under this section.

Testing Procedures

To test this program, I followed a cycle of running the program, checking for possible errors or inaccuracies, and changing the code accordingly. Some problems such as the program's inability to detect the plus and minus keys on the game screen were persistent and required multiple trials.

Final Build

The button from the prototype model was initially used to change the mode but its tasks have been replaced with the use of the "MODE" button.

The final code can also be found [here](#)

```
from machine import Pin, I2C, ADC

import time

import random

from ssd1306 import SSD1306_I2C

from ir_rx.nec import NEC_8

from ir_rx.print_error import print_error


i2c_left = I2C(0, scl=Pin(13), sda=Pin(12), freq=400000)

i2c_right = I2C(1, scl=Pin(15), sda=Pin(14), freq=400000)

oled_left = SSD1306_I2C(128, 64, i2c_left)

oled_right = SSD1306_I2C(128, 64, i2c_right)

joystick = ADC(27)

ir_pin = Pin(17, Pin.IN)
```

```
ir_button = None

def ir_callback(data, addr, ctrl):

    global ir_button

    if data >= 0:

        ir_button = data

def read_ir():

    global ir_button

    last_value = ir_button

    ir_button = None

    return last_value

ir = NEC_8(ir_pin, ir_callback)

#ir.error_function(print_error)

MODE = 0x46

OK = 0x45

PLUS = 0x09

MINUS = 0x15

WIDTH = 256

HEIGHT = 64
```

```
PADDLE_HEIGHT = 12

PADDLE_WIDTH = 2

BALL_SIZE = 2


game_started = False

ai_mode = False

winner = 0

score1 = 0

score2 = 0

paddle1_y = 26

paddle2_y = 26

ball_x = 128

ball_y = 32

ball_dx = 1

ball_dy = 1


def clear_displays():

    oled_left.fill(0)

    oled_right.fill(0)


def draw_pixel(x, y):

    if x < 0 or x >= WIDTH:

        return

    if y < 0 or y >= HEIGHT:

        return
```

```

    if x < 128:

        oled_left.pixel(x, y, 1)

    else:

        oled_right.pixel(x - 128, y, 1)


def draw_rect(x, y, w, h):

    for yy in range(y, y+h):

        for xx in range(x, x+w):

            draw_pixel(xx, yy)


def draw_text(x, y, text):

    if x < 128:

        oled_left.text(text, x, y, 1)

    else:

        oled_right.text(text, x-128, y, 1)


def show():

    oled_left.show()

    oled_right.show()


# -----
# DRAW FULL PLAYFIELD
# -----

```

```

def draw_field():

    clear_displays()

    # middle line

    for y in range(0,64,4):

        draw_pixel(128, y)

        draw_pixel(127, y)


    # paddles

    draw_rect(4, paddle1_y, PADDLE_WIDTH, PADDLE_HEIGHT)


    draw_rect(250, paddle2_y, PADDLE_WIDTH, PADDLE_HEIGHT)

    # ball

    draw_rect(ball_x, ball_y, BALL_SIZE, BALL_SIZE)


    # scores

    draw_text(55, 0, str(score1))

    draw_text(183, 0, str(score2))

    show()


# -----

# INPUT FUNCTIONS

# -----


def read_joystick():

```

```

value = joystick.read_u16()

# joystick centered around ~32768

if value < 20000:

    return -1


if value > 45000:

    return 1


return 0


# -----
# PADDLE CONTROL
# -----

def move_paddle1():

    global paddle1_y

    direction = read_joystick()

    paddle1_y += direction * 5


if paddle1_y < 0:

    paddle1_y = 0


if paddle1_y > HEIGHT - PADDLE_HEIGHT:

    paddle1_y = HEIGHT - PADDLE_HEIGHT

```

```

def move_paddle2_manual():

    #print("move_paddle2_manual() was called")

    global paddle2_y

    code = read_ir()

    #if code is not None:

        #print(f"From paddle2_manual: {code}")

    #if code is None:

        #    return

    if code == PLUS:

        #print("Plus detected!")

        paddle2_y -= 5

    elif code == MINUS:

        #print("Minus detected!")

        paddle2_y += 5

    if paddle2_y < 0:

        paddle2_y = 0

    if paddle2_y > HEIGHT - PADDLE_HEIGHT:

        paddle2_y = HEIGHT - PADDLE_HEIGHT

    #time.sleep(0.03)

```

```

# -----

# AI PADDLE

# -----

def move_paddle2_ai():

    global paddle2_y

    target = ball_y - (PADDLE_HEIGHT // 2)

    if paddle2_y < target:

        paddle2_y += 3

    elif paddle2_y > target:

        paddle2_y -= 3

    if paddle2_y < 0:

        paddle2_y = 0

    if paddle2_y > HEIGHT - PADDLE_HEIGHT:

        paddle2_y = HEIGHT - PADDLE_HEIGHT

# -----

# MENU SCREEN

# -----

def draw_menu():

```

```
clear_displays()

draw_text(50, 9, "PING")

draw_text(178, 9, "PONG")


if ai_mode:

    draw_text(5, 30, "VS BOT")


else:

    draw_text(5, 30, "VS PLAYER")


draw_text(5, 50, "POWER TO START")

draw_text(130, 30, "'MODE' TO CHANGE")

draw_text(130, 50, "GAMEMODE (P2/AI)")


show()


def menu():

    global ai_mode

    global game_started

    draw_menu()

    while not game_started:

        code = read_ir()
```

```
    if code == MODE:

        #print("MODE matched!")

        ai_mode = not ai_mode

        draw_menu()

    elif code == OK:

        #print("OK matched!")

        game_started = True

    time.sleep(1)

    # -----

# BALL FUNCTIONS

# -----

def reset_ball():

    global ball_x

    global ball_y

    global ball_dx

    global ball_dy

    ball_x = 128

    ball_y = 32
```

```
ball_dx = random.choice([-1, 1])

ball_dy = random.choice([-1, 1])


def reset_game():

    global score1

    global score2

    global paddle1_y

    global paddle2_y

    global winner

    score1 = 0

    score2 = 0

    paddle1_y = 26

    paddle2_y = 26

    winner = 0

    reset_ball()

# -----

# BALL COLLISION

# -----

def move_ball():

    global ball_x

    global ball_y

    global ball_dx

    global ball_dy
```

```

ball_x += ball_dx * 3

ball_y += ball_dy * 3

# Top and bottom walls

if ball_y <= 0:

    ball_y = 0

    ball_dy *= -1

if ball_y >= HEIGHT - BALL_SIZE:

    ball_y = HEIGHT - BALL_SIZE

    ball_dy *= -1

# Player 1 paddle

if ball_x <= 6:

    if (paddle1_y <= ball_y <= paddle1_y + PADDLE_HEIGHT):

        ball_x = 6

        ball_dx *= -1

# Player 2 paddle

if ball_x >= 246:

    if (paddle2_y <= ball_y <= paddle2_y + PADDLE_HEIGHT):

        ball_x = 246

        ball_dx *= -1

# -----

```

```
# SCORE HANDLING

# -----

def check_score():

    global score1

    global score2

    global winner

    if ball_x < 0:

        score2 += 1

        reset_ball()

    if ball_x > WIDTH:

        score1 += 1

        reset_ball()

    if score1 >= 5:

        winner = 1

    if score2 >= 5:

        winner = 2

# -----

# WIN SCREEN

# -----

def draw_winner():

    clear_displays()

    if winner == 1:
```

```
        draw_text(20, 25, "P1 WINS")

    else:

        draw_text(150, 25, "P2 WINS")

    show()

# -----
# MAIN GAME LOOP
# -----

def main():

    global game_started

    while True:

        game_started = False

        menu()

        reset_game()

        while game_started and winner == 0:

            # Player 1

            move_paddle1()

            # Player 2
```

```
if ai_mode:

    move_paddle2_ai()

    x = 0.01

else:

    move_paddle2_manual()

    x = 0.15


move_ball()

check_score()

draw_field()

time.sleep(x)


if winner != 0:

    draw_winner()

    time.sleep(5)

    reset_game()

    game_started = False


if __name__ == "__main__":

    main()
```

Conclusion

Overall, I am extremely satisfied with the conclusion of this project and hope to use the skills learned in this camp's series of small and large projects to help me make larger programs in the future.

